

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)** Department of Computer Science and
Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	M.S.Dinesh	Reg. No.:	192424016
Section / Batch:	I	Date:	27.08.26

Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems. • Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – Pseudocode Writing Task

Code of Conduct

I (M.S.Dinesh /192424016) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___M.S.Dinesh___

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

			Pseudocode Structure	10%	
Criteria	Wt.	Level Exemplary	Completeness	20%	
Problem Understanding	20%		Total Marks (100):		
Algorithm Design	30%				
Use of Control Structures	20%				

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Problem Definition

- **Input:**
 - **elements:** An array of n distinct items, $[e_0, e_1, \dots, e_{n-1}]$.
 -
 - **restrictions:** A mapping where key pos ($0 \leq pos < n$) contains the set of forbidden elements for that index.
 -
- **Output:** A list containing all valid permutations of length n that satisfy all position constraints.
- **Constraints:**
 - Each element must be used exactly once in every permutation.
 -
 - No element e can appear at index pos if $e \in \text{restrictions}[pos]$.
 -

Algorithmic Approach & Complexity Class

- **Approach: Backtracking (Depth-First Search).** Build permutations element by element. At each depth (position), prune invalid branches immediately if an element violates position restrictions or has already been used.
- **Complexity Class:**
 - **Decision Problem:** Belongs to class **NP** because checking whether a given permutation satisfies the constraints takes polynomial time $O(n)$.
 -
 - **Enumeration Problem:** Belongs to class **NP-Hard** in the general case, as it generalizes restricted permutation problems (equivalent to computing the permanent of a 0-1 matrix).
 -

State-Space Representation & Pruning Rules

- **State:** Represented by $(\text{current_permutation}, \text{used_flags}, \text{current_pos})$, where:
 1. **current_permutation** is the partial assignment of elements.
 2. **used_flags** is a boolean array of size n tracking assigned elements.
 3. **current_pos** represents the current position index (recursion depth).
- **Base Case:** When $\text{current_pos} == n$, record the valid permutation.

- **Pruning Conditions:**

1. *Already Used:* Skip element i if `used[i] == True`.
2. *Position Restriction:* Skip element i if `elements[i] ∈ restrictions[current_pos]`.

Pseudocode

Plaintext

Input: List 'elements' of size n , Dictionary 'restrictions'

Output: List 'results' of valid permutations

1. Initialize $n \leftarrow \text{length}(\text{elements})$
2. Initialize results $\leftarrow$ empty list
3. Initialize used $\leftarrow$ array of size n with all False
4. Initialize current_permutation $\leftarrow$ empty list
5. Function Backtrack(current_pos):
6. If current_pos == n :
7. Append a copy of current_permutation to results
8. Return
- 9.
10. For i from 0 to $n - 1$:
11. // Pruning Step 1: Check if already placed
12. If used[i] == False:
13. // Pruning Step 2: Check forbidden position constraint
14. If elements[i] NOT IN restrictions[current_pos]:
15. // Choose
16. used[i] $\leftarrow$ True
17. Append elements[i] to current_permutation
- 18.
19. // Explore
20. Backtrack(current_pos + 1)
- 21.
22. // Backtrack (Unchoose)
23. used[i] $\leftarrow$ False
24. Remove last element from current_permutation
25. Backtrack(0)
26. Return results

Step-by-Step Justification & Trace

- **Example:** `elements = ['A', 'B', 'C'], restrictions = {0: ['A']}` (Element 'A' cannot be at position 0).
- **Execution Trace:**
 1. **Position 0**
 - Try 'A': Blocked by restriction $\rightarrow$ **Pruned**.
 -
 - Try 'B': Valid $\rightarrow$ Pick 'B', move to Position 1.
 -
 2. **Position 1 (Prefix: ['B'])**
 - Try 'A': Valid $\rightarrow$ Pick 'A', move to Position 2.
 - **Position 2 (Prefix: ['B', 'A']):** Try 'C' $\rightarrow$ Valid $\rightarrow$ Solution: ['B', 'A', 'C'].
 - Try 'C': Valid $\rightarrow$ Pick 'C', move to Position 2.
 - **Position 2 (Prefix: ['B', 'C']):** Try 'A' $\rightarrow$ Valid $\rightarrow$ Solution: ['B', 'C', 'A'].
 3. **Position 0 (backtrack to root):**
 - Try 'C': Valid $\rightarrow$ Explores subtree yielding ['C', 'A', 'B'] and ['C', 'B', 'A'] (if not restricted).

Complexity Analysis

- **Time Complexity:**
 - **Worst Case:** $O(n \cdot n!)$ when no restrictions are applied (explores all permutations and spends $O(n)$ copying each solution).
 -
 - **Best/Pruned Case:** $O(\Omega(n))$ if constraints eliminate all paths at early depths.
- **Space Complexity:**
 - **Auxiliary Space:** $O(n)$ for the recursion call stack and tracking arrays (`used`, `current_permutation`).
 - **Total Space:** $O(k \cdot n)$ where k is the total number of valid permutations stored in the final output.

CO5 AT2-Pseudocode Writing Task (Algorithm Design)

← Back

Language: Python C C++ Java

QUESTIONS

Q1

Permutation Generation with
Position Restrictions
100 marks

1/1 answered

Q1 Permutation Generation with Position Restrictions

100 marks

Write a pseudocode using Backtracking to generate all permutations in which certain elements are not allowed in specified positions. Clearly define inputs (set of elements, restriction list) and output (valid permutations only). Design an algorithm that builds permutations one position at a time while checking position constraints. Use loops, recursive branching, and condition checks effectively during placement. Apply pruning to reject invalid assignments before deeper recursion. Present the pseudocode in a clean and structured format. Ensure the solution is complete and avoids generating invalid or duplicate permutations.

Your Code

```
1 def generate_restricted_permutations(elements, restrictions):
2     n = len(elements)
3     results = []
4     used = [False] * n
5     current_permutation = []
6
7     def backtrack():
8         if len(current_permutation) == n:
9             results.append(list(current_permutation))
10            return
11
12            current_pos = len(current_permutation)
13
14            for i in range(n):
15                if not used[i]:
16                    if elements[i] not in restrictions.get(current_pos, []):
17
18                        used[i] = True
19                        current_permutation.append(elements[i])
```

Input

stdin input

Output

```
[[ 'B', 'A', 'C'], [ 'B', 'C', 'A'],
[ 'C', 'A', 'B']]
```

Power & B

Energy saver
Windows is h